

SOFTWARE REQUIREMENTS SPECIFICATION

Verified Community Directory PWA

Secure MVP - Implementation Specification

A build-ready specification for secure identity, trusted devices, private directory data, shared communication, and administrative control.

SECURE APIs	SHARED RECORDS	AUDITABLE
-------------	----------------	-----------

Version 1.0 | Draft for implementation planning and review
Prepared for the Community Directory Project

Document Control

A concise implementation reference for engineering, QA, security review, and administrators.

Item	Detail
Document	Software Requirements Specification (SRS)
Product	Verified Community Directory PWA
Version	1.0 - Secure MVP
Status	Draft for implementation planning
Companion document	Product Requirements Document (PRD), Version 1.0
Primary implementation objective	Replace browser-side trust with backend-controlled identity, data access, communication, and auditability
Primary stakeholders	Project team, authorised administrators, community representatives, QA reviewers

Purpose

This SRS translates the approved product vision into buildable software requirements. It defines the system boundary, technical architecture, roles, functional requirements, interfaces, data model, security controls, non-functional requirements, acceptance criteria, and delivery sequence for the secure MVP.

Scope discipline

This document specifies the secure rebuild. The existing static PWA remains a UI reference only; it is not the production architecture. Public Google Sheet fetches, client-generated Member IDs, localStorage-only communication, and browser-only validation are prohibited in the target system.

Table of Contents

A quick map of the system requirements and engineering decisions.

- 1. Introduction and System Scope
- 2. Overall System Description
- 3. Architecture and Component Responsibilities
- 4. Users, Roles, and Access Control
- 5. Functional Requirements
- 6. External Interface Requirements
- 7. Data Requirements and Data Model
- 8. Security, Privacy, and Audit Requirements
- 9. Non-Functional Requirements
- 10. API Requirements and Error Conventions
- 11. Acceptance Criteria and Test Scenarios
- 12. Delivery Plan and Traceability
- Appendix A. Requirement ID Index
- Appendix B. Glossary

1. Introduction and System Scope

1.1 System purpose

The Verified Community Directory PWA is a mobile-first, installable web application for a private community. It enables approved members to discover authorised profiles, access permitted contact and professional information, share community updates, raise emergency alerts, and use the platform only through verified accounts and approved devices.

1.2 In-scope capabilities

- Admin-controlled member import, review, approval, activation, suspension, and deactivation.
- Backend-generated, non-guessable Member IDs issued only after approval.
- Registered-email OTP verification, secure server-side sessions, and passkey/WebAuthn trusted-device enrolment.
- Authenticated directory search, filters, profile views, and role- and consent-aware privacy projections.
- Centrally stored Inbox posts and Emergency alerts with optional private media attachments.
- Admin moderation, device-change approvals, suspicious-activity events, security flags, and audit logs.
- Installable PWA application shell with safe offline behavior.

1.3 Out-of-scope for the MVP

Excluded capability	Reason / later phase direction
Native Android/iOS application	The MVP remains a browser-installable PWA; APK packaging can be evaluated later.
IMEI or hardware-serial tracking	Browsers and PWAs cannot reliably access IMEI; trusted devices use passkeys, server records, and secure cookies.
Public or anonymous directory browsing	All directory data is private and backend-gated.
Private one-to-one chat, calls, or social feed algorithms	Not required for the core directory, Inbox, and Emergency use cases.
WebSocket platform / push notification infrastructure	MVP uses controlled polling; real-time delivery can be evaluated later.
Automated live Google Sheet as production datastore	Sheets may remain an intake source but never a public live source.

2. Overall System Description

2.1 Product context

Admin import source	Private database	Secure backend APIs	Verified PWA	Shared community use
---------------------	------------------	---------------------	--------------	----------------------

The system replaces a static frontend that directly fetched a Google Sheet and stored messages in browser localStorage. The target application has a central backend responsible for validating access, serving permitted

profile data, persisting communication records, issuing secure sessions, tracking trusted devices, and recording operational events.

2.2 Operating principles

Principle	Requirement implication
Backend is the source of truth	No user, role, Member ID, post, alert, device, or permission decision is trusted merely because the frontend says so.
Identity is session-derived	A typed Member ID is never treated as proof of identity; post authorship comes from the verified session.
Privacy by minimum disclosure	The member profile API returns only permitted fields for the requester role and member visibility settings.
Device approval is revocable	A member can be suspended, a device can be revoked, and sessions can be invalidated immediately by administrators.
Communication is central and auditable	Inbox and Emergency records are stored in the database, can be moderated, and include an activity trail.

2.3 Deployment assumptions

- The frontend and backend are deployed over HTTPS only.
- The database is private and not directly exposed to browsers.
- Email OTP delivery is available through an SMTP-compatible service account.
- Private media storage supports S3-compatible signed upload/download flows or an equivalent protected object-store interface.
- Administrators use supported modern browsers for passkey and dashboard workflows.

3. Architecture and Component Responsibilities

React PWA	FastAPI API layer	Domain services	MongoDB records	Media + Email
--------------	----------------------	--------------------	--------------------	------------------

3.1 Target architecture

Component	Recommended technology	Responsibility
Frontend	React + TypeScript + Vite	Mobile-first user experience, PWA shell, API consumption, local UI state only.
PWA layer	vite-plugin-pwa / Workbox	Manifest, icons, standalone installability, safe static asset caching.
Backend API	Python FastAPI	Authentication, authorization, directory, posts, alerts, admin operations, validation, and audit hooks.
Database	MongoDB Atlas / private MongoDB	Members, devices, credentials, sessions, OTP challenges, posts, flags, audit records, imports.

Component	Recommended technology	Responsibility
Authentication	Server-side opaque sessions + WebAuthn	Secure session issuance, passkey validation, trusted-device enforcement, revocation.
Email OTP	SMTP adapter	Registered-email one-time code delivery with rate-limit and expiry controls.
Media storage	Private S3-compatible object storage	Private images/videos with server-authorized access paths.
Deployment	Dockerised backend + static frontend host	Repeatable deployment, isolated environment variables, HTTPS termination.

3.2 Backend modules

Module	Core responsibility
auth	Identity initiation, OTP challenge creation, OTP verification, session lifecycle, logout.
webauthn	Registration/login options, verification, credential management, signature counter updates.
devices	Trusted-device records, device-change requests, approval/rejection, revocation.
members	Directory query, privacy projection, profile retrieval, member status handling.
posts	Inbox posts, Emergency alerts, reporting, moderation, resolution workflow.
media	Upload intent, validation, storage confirmation, access URL issuance.
admin	Imports, member approval, moderation, exports, security reviews.
security	Rate limits, security-event creation, threshold evaluation, flags, audit logging.

Architectural rule

The frontend may format and display information, but it must not be the authorization boundary. Every protected backend request re-validates the active session, member state, device state, and required role.

4. Users, Roles, and Access Control

Role	Allowed actions	Restricted actions
Visitor / unauthenticated	Open public landing, identify for enrolment/login, view generic help	No directory, profile, post, alert, media, or admin access

Role	Allowed actions	Restricted actions
Pending member	Complete approved enrolment workflow	No directory or feed access until device activation succeeds
Active member	View permitted profiles, search/filter directory, post Inbox/Emergency, report content, request device change	No member approval, moderation, flags, audit logs, or admin exports
Administrator	Manage members, imports, devices, posts, alerts, flags, reports, and privacy settings	No direct access to raw session token values, OTP values, or cryptographic secret material
System maintainer	Deployment, configuration, backups, monitoring, secure admin bootstrap	No routine content moderation unless explicitly assigned Admin role

4.1 Access decision model

Valid session	Active member	Active device	Required role	Privacy projection	API response
---------------	---------------	---------------	---------------	--------------------	--------------

The access decision must be enforced on every protected API request. A prior successful screen navigation does not grant permanent authority.

5. Functional Requirements

All requirements in this section are mandatory for the secure MVP unless marked as later-phase.

5.1 Administration and bootstrap

ID	Requirement
FR-ADM-001	The system shall create the first administrator only through environment-variable bootstrap configuration.
FR-ADM-002	The system shall not include hardcoded administrator credentials, passwords, OTP values, JWT secrets, or reset tokens in source code.
FR-ADM-003	The system shall record bootstrap completion and shall not recreate or reset the bootstrap administrator on restart.
FR-ADM-004	The system shall deny protected Admin APIs to users without the Admin role.
FR-ADM-005	The system shall record member edits, approvals, suspensions, device decisions, moderation, exports, and configuration changes in audit logs.
FR-ADM-006	The Admin Dashboard shall support member create, edit, approve, activate, suspend, deactivate, and export operations.

5.2 Member import, approval, and IDs

ID	Requirement
FR-IMP-001	The system shall allow an Admin to upload a CSV containing proposed member records.
FR-IMP-002	Imported rows shall enter STAGED status and shall not automatically become visible in the directory.
FR-IMP-003	The import service shall validate required fields, normalise email/phone fields, and flag malformed or duplicate records.
FR-IMP-004	The system shall preserve import-run metadata, source filename, uploader identity, timestamp, row status, and validation errors.
FR-IMP-005	On approval, the backend shall generate a unique random Member ID that is not derived from a name, timestamp, or browser hash.
FR-IMP-006	Approved members shall enter PENDING_ENROLLMENT status until verified enrolment is complete.

5.3 Authentication and OTP

ID	Requirement
FR-AUTH-001	Unauthenticated users shall not access directory, profile, posts, alerts, media, devices, or Admin APIs.
FR-AUTH-002	The enrolment identity step shall require a Member ID and registered email address.
FR-AUTH-003	Identity responses shall be generic and shall not confirm whether any individual Member ID or contact value exists.
FR-AUTH-004	The system shall send an OTP only when submitted details match an approved eligible record.
FR-AUTH-005	OTP validity shall be configurable within a 5-10 minute window and shall allow no more than five verification attempts per challenge.
FR-AUTH-006	OTP resend shall enforce a configurable cooldown of 30-60 seconds and a daily request cap.
FR-AUTH-007	OTP values shall be stored only as one-way hashes and shall be invalidated after successful consumption or expiry.
FR-AUTH-008	The system shall record failed identity checks, invalid OTP attempts, resend abuse, and verification success events.

5.4 Sessions and trusted devices

ID	Requirement
FR-DEV-001	The system shall issue opaque server-side sessions through Secure, HttpOnly, SameSite cookies only.

ID	Requirement
FR-DEV-002	The browser shall not store sessions, access tokens, or identity claims in localStorage, sessionStorage, query strings, or frontend configuration.
FR-DEV-003	First-time activation shall require a WebAuthn/passkey registration response that is verified by the backend.
FR-DEV-004	The backend shall validate WebAuthn challenge, origin, relying-party ID, credential ID, public key, signature, and user verification requirements.
FR-DEV-005	A successful first-time activation shall create an ACTIVE Trusted Device record and a secure trusted-device cookie.
FR-DEV-006	Future login shall require a valid passkey assertion, an ACTIVE device record, an account in ACTIVE status, and a matching trusted-device cookie.
FR-DEV-007	A new phone, browser, or cleared cookie state shall create a PENDING device-change request after OTP verification and shall not grant directory access automatically.
FR-DEV-008	On approval of a device change, the backend shall revoke old device records and all sessions associated with the replaced device.
FR-DEV-009	The system shall log invalid trusted-device tokens, repeated failed passkey assertions, device revocations, and device-change decisions.

5.5 Directory and profile access

ID	Requirement
FR-DIR-001	The directory API shall return results only for ACTIVE members with a valid session and ACTIVE trusted device.
FR-DIR-002	The system shall support query search by permitted fields including name, city, state, organisation, occupation, education, and blood group.
FR-DIR-003	The system shall support configurable filters for State, Blood Group, Occupation, Education Sector, and Sub-Caste where privacy policy permits.
FR-DIR-004	Directory list responses shall be paginated and shall return only the card-level profile fields required by the current view.
FR-DIR-005	Member profile responses shall use backend-controlled privacy projections based on viewer role and target member visibility settings.
FR-DIR-006	The frontend shall not request or receive an unrestricted raw directory dump or a public Google Sheet CSV endpoint.
FR-DIR-007	No real member data shall be embedded as frontend fallback data, service-worker seed data, or static source code fixtures in production.

5.6 Inbox and Emergency communication

ID	Requirement
FR-COM-001	Only ACTIVE members with valid session and device state shall create Inbox posts and Emergency alerts.
FR-COM-002	The backend shall derive post author identity from the session; the frontend shall not request a Member ID for posting.
FR-COM-003	Inbox posts and Emergency alerts shall be stored in the central database and returned through authenticated feed APIs.
FR-COM-004	Each communication record shall include type, author, message, optional media references, status, timestamps, and moderation metadata.
FR-COM-005	Emergency alerts shall be marked URGENT, sorted with priority in the Emergency feed, and support ACTIVE and RESOLVED states.
FR-COM-006	Admins shall be able to remove/hide a post with a reason and to resolve an Emergency alert with an optional resolution note.
FR-COM-007	Members shall be able to report posts for Admin review.
FR-COM-008	Feed polling may be used in MVP; Inbox refresh interval shall not be more frequent than 45 seconds and Emergency polling shall not be more frequent than 15 seconds while the relevant view is open.

5.7 Media

ID	Requirement
FR-MED-001	The system shall support authorised image and video attachments for Inbox and Emergency records.
FR-MED-002	The backend shall validate upload authorisation, media type, file size, metadata, and final ownership before linking media to a post.
FR-MED-003	The MVP shall limit image uploads to 5 MB and video uploads to 20 MB unless administrators configure lower limits.
FR-MED-004	Media shall be stored privately and accessed only through backend-authorised time-limited URLs or an equivalent protected stream.
FR-MED-005	The system shall reject unsupported or suspicious media uploads and record the event.

5.8 Security events and flags

ID	Requirement
FR-SEC-001	The system shall create security events for wrong Member ID/email attempts, invalid OTPs, invalid device tokens, failed WebAuthn assertions, and unauthorised posting attempts.

ID	Requirement
FR-SEC-002	The system shall rate-limit repeated authentication and OTP misuse by request source and challenge context.
FR-SEC-003	The system shall create Admin-visible Security Flags when configured thresholds are reached.
FR-SEC-004	The system shall not automatically lock a target member account solely because an unauthenticated actor repeatedly enters that member ID incorrectly.
FR-SEC-005	Admins shall be able to review, resolve, annotate, suspend, revoke, or export supported security records according to role.

6. External Interface Requirements

6.1 User interface requirements

UI area	Required behavior
Public / entry screen	Shows generic sign-in/enrolment entry without exposing member existence or directory data.
OTP screen	Provides masked email destination where appropriate, countdown, resend cooldown, attempt feedback without leaking implementation details.
Passkey screen	Requests browser-native passkey flow and clearly explains unsupported-device outcome.
Directory screen	Shows paginated cards, search, enabled filters, loading/empty/error states, and no data before authentication.
Profile sheet	Displays only fields returned by the backend privacy projection; must not infer hidden fields from client data.
Inbox / Emergency	Shows shared feed, visible author derived from backend, safe media previews, report and moderation states.
Admin dashboard	Separates member management, imports, devices, alerts, security flags, audit views, and moderation actions.

6.2 Backend API interface

Endpoint group	Required endpoints
Authentication	POST /api/auth/identify, POST /api/auth/otp/verify, GET /api/auth/me, POST /api/auth/logout
WebAuthn	POST /api/auth/webauthn/register/options, POST /api/auth/webauthn/register/verify, POST /api/auth/webauthn/login/options, POST /api/auth/webauthn/login/verify
Devices	GET /api/devices/current, POST /api/devices/change-request,

Endpoint group	Required endpoints
	DELETE /api/devices/current
Directory	GET /api/members, GET /api/members/filters, GET /api/members/{member_id}
Communication	GET/POST /api/posts, POST /api/posts/{post_id}/report, GET/POST /api/emergency-alerts, POST /api/emergency-alerts/{alert_id}/resolve
Media	POST /api/media/upload-intent, POST /api/media/complete, GET /api/media/{media_id}/access-url
Administration	POST /api/admin/imports/members, PATCH /api/admin/members/{member_id}, POST /api/admin/members/{member_id}/approve, POST /api/admin/device-change-requests/{request_id}/approve, GET /api/admin/security-flags

6.3 Integration interfaces

Integration	Required contract
Google Form / Google Sheet	May act only as an Admin-controlled intake source. CSV export/import is the MVP ingestion path; no public CSV endpoint is consumed by the PWA.
Email provider	SMTP-compatible adapter capable of sending OTP templates, logging delivery attempts, and operating through environment variables.
Object storage	Private bucket/container, scoped upload intents, server-side ownership checks, signed access URL generation or protected proxy.
Browser WebAuthn API	Browser supports credential creation and assertion; backend verifies cryptographic responses.

7. Data Requirements and Data Model

7.1 Core collections

Collection	Key fields	Purpose
members	member_id, email, role, status, profile, visibility_settings	Approved member record and privacy controls
devices	member_id, status, trusted_device_cookie_hash, credential reference	Trusted-device lifecycle and revocation
webauthn_credentials	credential_id, public_key, sign_count, member_id, device_id	Passkey verification material
sessions	session_token_hash, member_id, device_id, expires_at, revoked_at	Server-side session control

Collection	Key fields	Purpose
otp_challenges	member_id, purpose, otp_hash, attempts, expires_at, consumed_at	Short-lived OTP verification challenges
posts	type, author_member_id, message, media_ids, status, timestamps	Inbox messages and Emergency alerts
media_objects	owner, storage_key, content_type, bytes, linked_post_id	Private uploaded-media metadata
security_events	event_type, source hash, member/device reference, timestamp	Low-level failed/suspicious actions
security_flags	status, severity, event references, admin notes	Admin-reviewable aggregated incidents
audit_logs	actor, action, target, before/after summary, timestamp	Accountability for sensitive operations
import_runs	uploader, source file, rows, statuses, validation results	CSV import history and review trace

7.2 Required member status values

STAGED	PENDING ENROLLMENT	ACTIVE	SUSPENDED	DEACTIVATED
--------	--------------------	--------	-----------	-------------

Only ACTIVE members may access the directory, create posts, or raise alerts. A device state is independently evaluated and must also be ACTIVE.

7.3 Required indexes

Index	Type	Purpose
members.member_id	Unique	Resolve approved member record without collisions
members.registered_email_normalized	Unique	Prevent duplicate registered emails
webauthn_credentials.credential_id	Unique	Map a passkey credential to one trusted record
devices.trusted_device_cookie_hash	Unique	Match browser device cookie with active device record
sessions.session_token_hash	Unique	Look up opaque session safely
otp_challenges.expires_at	TTL	Remove expired OTP challenges
sessions.expires_at	TTL	Remove expired session documents
posts.type + posts.created_at	Compound	Efficient Inbox/Emergency chronological feeds
security_events.created_at	Descending	Fast operational incident review

7.4 Data retention and deletion

- OTP challenges and expired sessions are short-lived operational records and should be removed automatically using expiry/TTL controls.
- Posts and Emergency alerts are retained according to community policy; removed content is soft-deleted with moderation metadata unless legal policy requires hard deletion.
- Audit logs must not expose secret values and should be retained long enough for administrator accountability and incident review.
- Deactivated member records shall be removed from normal directory responses; data deletion and retention require an explicit community policy before production launch.

8. Security, Privacy, and Audit Requirements

8.1 Security controls

ID	Control
SEC-001	All production traffic shall use HTTPS. HTTP access shall redirect or fail safely.
SEC-002	CORS shall permit only configured trusted frontend origins.
SEC-003	All protected API endpoints shall validate session, account status, device status, and required role server-side.
SEC-004	Session and trusted-device cookie values shall be Secure, HttpOnly, and protected by a strict appropriate SameSite policy.
SEC-005	Passwords, OTP values, session tokens, device tokens, hashes, private keys, cloud credentials, and SMTP secrets shall never be logged or embedded in frontend code.
SEC-006	Inputs shall be schema-validated; user-generated text shall be safely encoded/sanitised before rendering.
SEC-007	Media upload validation shall enforce type, size, ownership, and authorization; public permanent Drive links are prohibited for private member media.
SEC-008	The system shall apply rate limits for identity checks, OTP generation, OTP verification, and suspicious request patterns.
SEC-009	Administrative actions shall create audit records containing actor, action, target, time, and outcome.
SEC-010	The service worker shall cache only safe app shell assets and shall not cache protected API payloads by default.

8.2 Privacy projection matrix

Data category	Verified member viewing another member	Admin
Directory card	Name, image/initials, city/state, approved summary fields	All approved card fields
Contact details	Only where target member permits	Available for approved administrative

Data category	Verified member viewing another member	Admin
	exposure	purpose
Education and profession	Permitted profile fields	Full approved member record
Blood group	Visible only if member policy enables it	Visible where stored
DOB, pincode, family, spouse/children, caste/sub-caste	Hidden by default unless explicit policy/consent allows	Admin-only by default
Device, security, OTP, audit records	Never visible	Admin only, subject to role and operational need

8.3 Security-event thresholds

Trigger	Immediate system behavior	Escalation
Five invalid OTP attempts on one challenge	Expire challenge and require a new OTP after cooldown	Record security event
Rapid OTP resend attempts	Enforce resend cooldown and request-source rate limit	Create flag if threshold persists
Repeated invalid device token / cookie	Deny access; do not expose why the token is invalid	Create security flag for Admin review
Repeated Member ID/email mismatch attempts	Return generic message and rate-limit request source	Flag repeated source pattern; do not lock target member
Repeated unsupported/malformed media uploads	Reject upload and retain safe metadata only	Admin-visible security event / flag

9. Non-Functional Requirements

Category	Requirement
Performance	At normal load, directory list and profile APIs should respond within 500 ms excluding media transfer.
Performance	Directory results must be paginated; APIs must not return the complete raw community dataset by default.
Availability	The frontend must display clear loading, offline, and retry states without exposing raw server errors.
Reliability	Member approval, device revocation, and moderation writes must fail safely and avoid partial activation where possible.
Maintainability	Frontend, API routes, domain services, repositories, models, and configuration must be modular; the production system shall not be a single monolithic HTML file.
Observability	Backend shall emit structured non-secret logs for request failures, important security events, and operational errors.
Accessibility	Interactive controls must support keyboard navigation, visible focus, readable labels, and mobile touch targets.

Category	Requirement
Compatibility	The MVP shall support current Chrome/Edge on desktop and Android, and Safari-compatible browser behavior where WebAuthn support permits.
Scalability	Initial target capacity: up to 5,000 member profiles, 100 concurrent active users, and 10,000 communication records per month.
Time	All server timestamps must be stored in UTC; user interfaces may format them in the user locale/time zone.
Backup	A documented database backup and restore procedure must exist before pilot deployment.

9.1 Development quality gates

Gate	Minimum evidence before phase acceptance
Code quality	Linting/formatting passes, environment variables documented, secrets excluded from repository.
API quality	Schema validation, role checks, error convention, and integration tests for protected endpoints.
Security quality	Negative-path tests for unauthenticated access, invalid OTP, new device, revoked device, and direct API attempts.
PWA quality	Installability test, logout behavior, fresh browser test, and cache review demonstrating no private API cache leakage.
Operational quality	Admin action audit trail, import failure handling, backup/restore rehearsal, and support runbook.

10. API Requirements and Error Conventions

10.1 API conventions

Area	Convention
Base path	All backend endpoints are grouped under /api.
Format	Request and response bodies use JSON except controlled media upload flows.
Authentication	Browser sends secure cookies automatically; JavaScript never manually reads session token values.
Pagination	List endpoints accept page and page_size with capped page_size limits.
Filtering	Directory filters use named query parameters and only enabled privacy-safe filter fields.
Idempotency	Sensitive create operations may support idempotency keys in later hardening; not a blocker for first MVP.

Area	Convention
Error payload	All errors return a stable error code, safe message, and correlation/request ID where available.

10.2 Standard safe error response

Example response body

```
{"error": {"code": "AUTHENTICATION_REQUIRED", "message": "Please sign in to continue.", "request_id": "..."}}
```

10.3 Authentication error behavior

Scenario	Client message	Server action
Member ID/email mismatch	If the submitted details match an approved member record, a verification code has been sent.	Record non-sensitive event; do not reveal existence.
Invalid or expired OTP	The code could not be verified. Please request a new code if needed.	Increment challenge attempts / expire as required.
No active trusted device	This device needs administrator approval before access can be granted.	Create or show pending device-change state after eligible OTP verification.
Revoked session/device	Your session has ended. Please sign in again or contact an administrator.	Deny protected API request and log safe operational event.
Forbidden Admin action	You do not have permission to perform this action.	Return 403 and record an authorization event where relevant.

11. Acceptance Criteria and Test Scenarios

Test ID	Scenario	Expected result
AT-001	Unauthenticated browser requests directory API	Request is denied and no member data is returned.
AT-002	User submits invalid Member ID/email pair	Generic response is returned; no account existence is disclosed.
AT-003	User enters five invalid OTPs on a challenge	Challenge expires, access is denied, and event is recorded.
AT-004	Approved member completes OTP but is on a new device	Device remains pending; directory is blocked until admin approval and passkey registration.
AT-005	Member logs in from active approved device	Valid WebAuthn assertion plus matching device record creates a session.
AT-006	Member creates Inbox message	Post is saved centrally and visible to another verified member.
AT-007	Member creates Emergency alert	Urgent alert appears in shared Emergency feed and is visible to Admin.

Test ID	Scenario	Expected result
AT-008	Admin resolves Emergency alert	Alert status changes to RESOLVED and audit log captures actor/time/note.
AT-009	Admin revokes trusted device	Associated sessions are invalidated and next protected request is denied.
AT-010	Admin removes inappropriate post	Post is hidden/soft-deleted with moderation reason and audit entry.
AT-011	Service worker/cache inspected after directory use	Private directory/profile API responses are not persisted as generic app-shell cache.
AT-012	Production frontend source inspected	No real fallback records, public Sheet endpoint, or sensitive secret values exist.
AT-013	Suspended member calls protected API	Request denied; no posts or profiles are returned.
AT-014	Member changes privacy setting	Subsequent profile API projection reflects allowed visibility only.
AT-015	Repeated invalid device token requests	Requests denied; security events recorded and flag generated at threshold.

11.1 Minimum release exit criteria

- All Must Have functional requirements have passing integration tests or documented manual evidence.
- Critical security paths have negative tests: unauthenticated access, wrong identity, OTP abuse, new device, revoked device, suspended user, and unauthorized admin action.
- No live public Google Sheet is required by the frontend at runtime.
- Inbox and Emergency records are proven to sync across two separate verified browser sessions.
- Admin audit trail is demonstrated for member approval, device approval/revocation, post moderation, and alert resolution.
- PWA installation, logout, fresh-browser access block, and safe cache behavior are verified on target mobile browsers.

12. Delivery Plan and Traceability

Phase	Deliverable	Primary requirements covered
0. Legacy freeze	Preserved UI reference, safe fake fixtures, no real frontend secrets/data	FR-DIR-006, FR-DIR-007, SEC-005
1. Backend foundation	FastAPI, MongoDB, config, bootstrap Admin, audit utility, health endpoint	FR-ADM-001 to FR-ADM-005
2. Import and member management	CSV staging, validation, approval, secure IDs, Admin dashboard core	FR-IMP-001 to FR-IMP-006
3. Protected directory	React migration, secure directory APIs, profile projections, filters	FR-DIR-001 to FR-DIR-007
4. Identity and devices	Email OTP, sessions, WebAuthn, trusted devices, change requests	FR-AUTH-001 to FR-AUTH-008; FR-DEV-001 to FR-DEV-009
5. Communication	Inbox, Emergency, private media, reporting, resolution, moderation	FR-COM-001 to FR-COM-008; FR-MED-001 to FR-MED-005
6. Security operations + PWA hardening	Flags, exports, test suite, safe caching, pilot readiness	FR-SEC-001 to FR-SEC-005; SEC-001 to SEC-010

12.1 Traceability rule

Every implementation ticket should reference at least one requirement ID in this SRS. Every acceptance test should map back to one or more functional/security requirements. Any scope change that affects identity, privacy, device trust, communication storage, or admin action must update this SRS before release.

Appendix A. Requirement ID Index

Prefix	Area
FR-ADM	Administration and bootstrap
FR-IMP	Member import, approval, and Member ID issuance
FR-AUTH	Identity verification, OTP, and sessions
FR-DEV	WebAuthn and trusted device management
FR-DIR	Directory and profile access
FR-COM	Inbox and Emergency communication
FR-MED	Private media uploads and access
FR-SEC	Security events, rate limits, flags, and review
SEC	Cross-cutting security controls
AT	Acceptance tests

Appendix B. Glossary

Term	Meaning
PWA	Progressive Web Application: an installable web app that can open from a phone home screen.
Member ID	Random backend-issued identifier for an approved member. It is not a password or proof of identity.
OTP	One-Time Password: short-lived verification code delivered to a registered contact method.
WebAuthn / passkey	Browser-supported public-key authentication using a device biometric, PIN, or secure screen lock.
Trusted device	Server-approved browser/device record linked to verified passkey credentials and a secure device cookie.
Opaque session	Random server-recognised session identifier in a secure cookie; browser code cannot inspect its claims.
Security event	Recorded failed or suspicious action such as invalid OTP, wrong identity, or invalid device attempt.
Security flag	Admin-reviewable incident created when a rule threshold is reached.
Audit log	Accountability record showing actor, action, target, time, and outcome for important changes.
Privacy projection	Backend-selected subset of profile fields returned based on role and visibility settings.

End of Software Requirements Specification